Chapter 1 — Engine & Grid Core

This chapter establishes the absolute foundation of the game. Every system in every subsequent chapter depends on at least one system defined here. Layer 00 builds the zero-dependency engine primitives — the state machine that governs what the game *is* doing at any moment, the input pipeline that feeds every player action into every system, the tile rendering infrastructure that makes the world visible, and the pathfinding graph that gives movement its spatial intelligence. Layer 01 builds the grid itself as an interactive space: the cursor that is the player's hand on the battlefield, the camera that frames the battlefield as a readable composition, and the terrain rules that give the grid strategic texture. Nothing in the game — no unit, no combat, no story trigger, no AI decision — can function without this chapter being complete and stable first. Chapters 2 through 7 are all downstream of what is specified here.

Layer 00 — Engine Foundation

Dependency: None. This layer has zero dependencies. It must be built first. All other layers depend on at least one system defined here.

SYS-05 — Game State Manager (Scene / Phase Transitions) | Cat 4 | Input & Core Engine

- [ ] **Global State Machine**
  - [ ] Maintain a single, authoritative global game state at all times. Only one state may be active at any moment.
  - [ ] Define the following discrete, named states as the complete set of valid top-level states:
    - [ ] `TITLE_SCREEN` — Game is displaying the title/main menu screen.
    - [ ] `MAIN_MENU` — Player is navigating the main menu (New Game, Continue, Load, Settings).

        - [ ] `CUTSCENE` — A pre-battle, post-battle, or mid-chapter story sequence is playing. All gameplay input is suppressed.
        - [ ] `PRE_BATTLE_PREP` — The deployment/preparation screen is active before a chapter begins.
        - [ ] `BATTLE_MAP` — The tactical battle map is active. This is the primary gameplay state.
        - [ ] `BATTLE_MAP_PAUSED` — The battle map is active but halted; the pause menu is open.
        - [ ] `COMBAT_ANIMATION` — A combat exchange animation is playing. Map interaction is suspended until animation completes.
        - [ ] `DIALOGUE` — An in-map dialogue event is playing (e.g., a support conversation, a recruitment talk, a mid-battle event trigger). Map cursor input is suppressed.
        - [ ] `CHAPTER_CLEAR` — The chapter has been won. The results/clear screen is displayed.
        - [ ] `GAME_OVER` — A game-over condition has been met. The game-over screen is displayed.
        - [ ] `SAVE_MENU` — The save/load interface is open.
        - [ ] `SETTINGS_MENU` — The settings/options screen is open.
    - [ ] Each state owns a defined set of systems that are active while that state is current. Systems belonging to inactive states must not process input or update game logic.
- [ ] **Transition Logic**
    - [ ] Define all valid state transitions explicitly. No transition not in the defined transition table may occur.
    - [ ] The following transitions must be supported:
        - [ ] `TITLE_SCREEN` → `MAIN_MENU` (player presses confirm / any button)
        - [ ] `MAIN_MENU` → `CUTSCENE` (New Game selected — opening cinematic fires)
        - [ ] `MAIN_MENU` → `PRE_BATTLE_PREP` (Continue/Load selected — chapter resumes at prep screen if applicable)
        - [ ] `MAIN_MENU` → `BATTLE_MAP` (Continue/Load selected — chapter resumes mid-battle if saved mid-chapter)

- [ ] CUTSCENE → PRE_BATTLE_PREP (opening cutscene ends — chapter prep begins)
- [ ] CUTSCENE → BATTLE_MAP (mid-chapter cutscene ends — control returns to map)
- [ ] PRE_BATTLE_PREP → BATTLE_MAP (player confirms deployment and begins battle)
- [ ] BATTLE_MAP → CUTSCENE (a scripted event trigger fires mid-battle)
- [ ] BATTLE_MAP → COMBAT_ANIMATION (a combat exchange is initiated)
- [ ] BATTLE_MAP → DIALOGUE (a dialogue event triggers on the map)
- [ ] BATTLE_MAP → BATTLE_MAP_PAUSED (player presses pause)
- [ ] BATTLE_MAP → CHAPTER_CLEAR (win condition met)
- [ ] BATTLE_MAP → GAME_OVER (game-over condition met)
- [ ] BATTLE_MAP_PAUSED → BATTLE_MAP (player unpauses or closes pause menu)
- [ ] BATTLE_MAP_PAUSED → SAVE_MENU (player opens save from pause menu)
- [ ] BATTLE_MAP_PAUSED → SETTINGS_MENU (player opens settings from pause menu)
- [ ] COMBAT_ANIMATION → BATTLE_MAP (animation sequence completes — returns to map)
- [ ] DIALOGUE → BATTLE_MAP (dialogue event ends — returns to map)
- [ ] CHAPTER_CLEAR → CUTSCENE (post-battle cutscene triggers)
- [ ] CHAPTER_CLEAR → SAVE_MENU (player is prompted to save after chapter)
- [ ] GAME_OVER → MAIN_MENU (player selects "Return to Main Menu")
- [ ] GAME_OVER → SAVE_MENU (player selects "Load" from game over screen)
- [ ] SAVE_MENU → BATTLE_MAP_PAUSED (save/load complete — returns to paused map)

        - [ ] SAVE_MENU → CHAPTER_CLEAR (save complete from post-chapter save prompt)
        - [ ] SAVE_MENU → MAIN_MENU (load selected from main menu context)
        - [ ] SETTINGS_MENU → BATTLE_MAP_PAUSED (settings closed — returns to paused map)
        - [ ] SETTINGS_MENU → MAIN_MENU (settings opened from main menu context)
    - [ ] Transitions must be atomic: the previous state must be fully exited (all its systems deactivated, all transient data cleared or handed off) before the new state is entered.
    - [ ] No transition may be triggered by gameplay code directly. All transition requests must be submitted through a single transition request interface. The State Manager validates the request against the legal transition table before executing it. Illegal transition requests must be rejected and logged as errors.
- [ ] **Phase Tracking within BATTLE_MAP**
    - [ ] The BATTLE_MAP state contains its own internal sub-state tracking the current combat phase. This is distinct from the top-level state machine but is owned and managed by the State Manager.
    - [ ] Valid battle phases:
        - [ ] PLAYER_PHASE — It is the player's turn. Player input is active.
        - [ ] ENEMY_PHASE — The enemy AI is executing its turn. Player input is suppressed.
        - [ ] ALLIED_PHASE — Allied NPC commanders are executing their turns. Player input is suppressed.
    - [ ] Battle phase transitions must follow the order: PLAYER_PHASE → ENEMY_PHASE → ALLIED_PHASE → PLAYER_PHASE (cycling). The allied phase only fires on chapters where allied units are present; if no allied units are present, the cycle is PLAYER_PHASE → ENEMY_PHASE → PLAYER_PHASE.
    - [ ] The current battle phase must be queryable by any system that needs it (e.g., input system, AI controller, UI).
- [ ] **State Entry and Exit Hooks**

    - [ ] Each state must define an entry procedure (runs once when the state is entered) and an exit procedure (runs once when the state is exited).
    - [ ] Entry procedures are responsible for: activating required systems, loading required assets, initialising transient state for that screen/phase.
    - [ ] Exit procedures are responsible for: deactivating systems, releasing transient resources, handing off any persistent state to the world state tracker.
- [ ] **Error and Fallback Handling**
    - [ ] If a requested transition is illegal (not in the transition table), the State Manager must: reject the transition, log an error message identifying the source and target states and the system that requested it, and leave the current state unchanged.
    - [ ] If the game state is ever undefined or null (e.g., corruption on load), the State Manager must default to `MAIN_MENU` and log a critical error.

EDGE CASES

- A combat animation is interrupted mid-sequence (e.g., by a game crash or an unexpected error): on reload, the state must recover to `BATTLE_MAP` at the start of the combat exchange that was interrupted, not mid-animation.
- A dialogue event fires simultaneously with a chapter-clear condition (e.g., killing the boss triggers a mid-battle conversation and the win state at the same moment): the dialogue must play to completion before transitioning to `CHAPTER_CLEAR`. The win condition is queued, not discarded.
- A save menu is opened from the chapter-clear screen and then a load is initiated: the load must correctly restore to the saved chapter context, not return to the chapter-clear screen.
- Multiple transition requests submitted in the same frame: only the first request in the frame is processed; all others are discarded and logged.

SYS-03 — Input Mapping & Controller Support | Cat 4 | Input & Core Engine

- [ ] **Input Device Support**
  - [ ] Support the following input devices simultaneously, with runtime hot-switching between them (the game detects which device was most recently used and updates prompts accordingly):
    - [ ] Keyboard (primary PC input)
    - [ ] Gamepad / Controller (XInput-compatible; DirectInput as fallback on PC)
    - [ ] Mouse input for cursor positioning and menu interaction (PC only)
  - [ ] At no time should the game require a specific device. All core gameplay functions must be reachable on keyboard alone, and on gamepad alone.
- [ ] **Logical Action Definitions**
  - [ ] All game inputs must be defined as named logical actions, not as raw device inputs. Game systems must only ever query logical actions, never raw key/button states.
  - [ ] Define the following logical actions as the complete set:
    - [ ] `CURSOR_UP`, `CURSOR_DOWN`, `CURSOR_LEFT`, `CURSOR_RIGHT` — Move the grid cursor one tile in the indicated direction.
    - [ ] `CONFIRM` — Confirm a selection, open a unit's action menu, select a menu item.
    - [ ] `CANCEL` — Cancel the current action, close a menu, deselect a unit.
    - [ ] `FAST_CURSOR` — Modifier held: increases cursor movement speed (reduces DAS repeat interval).
    - [ ] `OPEN_MAP_MENU` — Opens the map-level context menu (End Turn, Save, Settings, etc.) when no unit is selected.
    - [ ] `OPEN_UNIT_INFO` — Opens the Unit Info / Status Screen for the unit currently under the cursor.
    - [ ] `TOGGLE_MAP_OVERLAY` — Cycles through available map overlay views (terrain info, attack ranges, etc.).

        - [ ] PAUSE — Opens the pause menu during BATTLE_MAP state.
        - [ ] SKIP_ANIMATION — During COMBAT_ANIMATION state, skips to the end-of-combat result instantly.
        - [ ] SKIP_DIALOGUE — Advances or skips a line of dialogue during DIALOGUE or CUTSCENE state.
        - [ ] HOLD_ADVANCE_DIALOGUE — Held modifier: auto-advances dialogue lines at a steady pace without button-mashing.
        - [ ] NEXT_UNIT — In PLAYER_PHASE, moves the cursor to the next unacted player unit that still has an available action.
        - [ ] PREV_UNIT — In PLAYER_PHASE, moves the cursor to the previous unacted player unit.
- [ ] **Default Key/Button Bindings**
    - [ ] Keyboard defaults:
        - [ ] CURSOR_UP/DOWN/LEFT/RIGHT → Arrow keys AND W/A/S/D
        - [ ] CONFIRM → Z key AND Enter key
        - [ ] CANCEL → X key AND Backspace key
        - [ ] FAST_CURSOR → Left Shift (held)
        - [ ] OPEN_MAP_MENU → Enter key (when no unit selected) AND M key
        - [ ] OPEN_UNIT_INFO → I key
        - [ ] TOGGLE_MAP_OVERLAY → Tab key
        - [ ] PAUSE → Escape key AND P key
        - [ ] SKIP_ANIMATION → Space key
        - [ ] SKIP_DIALOGUE → Z key AND Enter key AND Space key
        - [ ] HOLD_ADVANCE_DIALOGUE → A key (held)
        - [ ] NEXT_UNIT → Right bracket] key
        - [ ] PREV_UNIT → Left bracket [key
    - [ ] Gamepad defaults (XInput layout):
        - [ ] CURSOR_UP/DOWN/LEFT/RIGHT → D-pad AND Left analog stick (with dead-zone filtering)
        - [ ] CONFIRM → A button (South)
        - [ ] CANCEL → B button (East)
        - [ ] FAST_CURSOR → Left Bumper (LB) held
        - [ ] OPEN_MAP_MENU → Start / Menu button

        - [ ] `OPEN_UNIT_INFO` → Y button (North)
        - [ ] `TOGGLE_MAP_OVERLAY` → Right Bumper (RB)
        - [ ] `PAUSE` → Start / Menu button
        - [ ] `SKIP_ANIMATION` → A button (South)
        - [ ] `SKIP_DIALOGUE` → A button (South)
        - [ ] `HOLD_ADVANCE_DIALOGUE` → B button (East) held
        - [ ] `NEXT_UNIT` → Right on D-pad (when in unit-cycle mode)
        - [ ] `PREV_UNIT` → Left on D-pad (when in unit-cycle mode)
- [ ] **Rebinding System**
    - [ ] Every logical action must be rebindable by the player. Rebinding is available through the Settings/Options menu.
    - [ ] Rebinding applies per-device: keyboard bindings and gamepad bindings are stored and edited independently.
    - [ ] Conflict detection: if the player attempts to bind a key/button already assigned to another action, the system must:
        - [ ] Display a warning indicating which action currently occupies that binding.
        - [ ] Offer the player the option to swap the two bindings or cancel the rebind.
        - [ ] Never allow two distinct actions to share the same physical input on the same device simultaneously.
    - [ ] A "Reset to Defaults" option must restore all bindings for the selected device to their factory defaults.
    - [ ] Custom bindings must persist between sessions (written to the player's settings save file).
- [ ] **Analog Stick Dead Zone (Gamepad)**
    - [ ] Apply a circular dead zone to both analog sticks. The default dead zone radius is 25% of maximum stick deflection.
    - [ ] Analog stick input below the dead zone threshold must produce zero logical input (no cursor movement registered).
    - [ ] Analog stick input above the dead zone threshold maps to the nearest cardinal direction (up/down/left/right). Diagonal input resolves to a single cardinal direction using

the dominant axis (the axis with greater deflection magnitude).
    - [ ] The dead zone value must be configurable via the Settings menu (range: 10%–50%).
- [ ] **Input Context Filtering**
    - [ ] Each game state defines a set of logical actions that are valid in that state. The input system must only fire actions valid for the current state.
    - [ ] Input received during `COMBAT_ANIMATION` must be buffered only for `SKIP_ANIMATION`. All other inputs during animation playback are discarded.
    - [ ] Input received during `CUTSCENE` must only pass `SKIP_DIALOGUE` and `HOLD_ADVANCE_DIALOGUE`. All other inputs are suppressed.
    - [ ] Input received during `ENEMY_PHASE` or `ALLIED_PHASE` (within `BATTLE_MAP`) must only pass `PAUSE` and `SKIP_ANIMATION` (if an animation is playing). All other map inputs are suppressed.
- [ ] **Mouse Input (PC Only)**
    - [ ] Mouse cursor position maps to the grid tile currently under the mouse cursor. Moving the mouse over a tile moves the grid cursor to that tile.
    - [ ] Left click on a tile is equivalent to `CONFIRM` on that tile.
    - [ ] Right click is equivalent to `CANCEL`.
    - [ ] The mouse pointer must be hidden while the gamepad or keyboard is the most recently used input device, and shown when the mouse is used.
    - [ ] When the mouse cursor moves off the game window (or the window loses focus), no mouse input must be processed until the mouse re-enters the window and focus is restored.

EDGE CASES

- Player presses `CONFIRM` and `CANCEL` on the same frame: `CANCEL` takes priority; `CONFIRM` is discarded.
- Gamepad is disconnected mid-session: the game must pause, display a "Controller disconnected" prompt, and remain paused until the controller is reconnected or the player switches to keyboard. Input from the keyboard must be accepted during this

paused state to allow the player to continue on keyboard if preferred.
- A rebind is made during a menu interaction: the rebind takes effect only after the settings menu is closed, not mid-navigation.
- NEXT_UNIT is pressed when all player units have already acted: the cursor must not move (no unit to cycle to). No error state.

ANM-03 — Map Tile Rendering & Tileset System | Cat 4 | Animation & Visual Feedback

- [ ] **Tile Grid Render Foundation**
  - [ ] Render the map as a 2D grid of uniformly sized tiles. Each tile occupies exactly one grid cell. No tile may span multiple cells.
  - [ ] Define a fixed logical tile size in pixels (e.g., 16×16 px). All rendering calculations must use this logical tile size. The logical tile size is the authoritative spatial unit for all grid coordinate math.
  - [ ] Support integer-scale rendering (1×, 2×, 3×, 4×) to allow the game to display at higher screen resolutions without sub-pixel artefacts. The scale factor is applied globally and uniformly.
  - [ ] The rendered tile grid must be pixel-perfect at every supported integer scale: no tile bleed, no gaps between tiles, no sub-pixel interpolation.
- [ ] **Tileset Asset Structure**
  - [ ] Define a tileset as a single sprite sheet image containing a fixed grid of individual tile frames.
  - [ ] Each tile in the sheet is identified by a unique integer tile ID corresponding to its position in the sheet (row-major order, starting from 0).
  - [ ] A map file references tile IDs, not tile images directly. The rendering system resolves tile IDs to pixel regions within the appropriate tileset sheet at render time.
  - [ ] Support multiple tileset sheets per map (i.e., a map may draw tiles from more than one tileset sheet). Each tile ID in the map data must specify both the sheet index and the local tile ID within that sheet.

    - [ ] Tileset sheets must support transparent pixels (alpha channel) to allow tile layering.
- [ ] **Tile Layers**
    - [ ] Each map supports at least the following render layers, drawn in strict back-to-front order:
        - [ ] **Layer 0 — Ground**: The base terrain tile (grass, stone, water, etc.). Drawn first; never transparent.
        - [ ] **Layer 1 — Overlay / Decoration**: Secondary terrain features layered on top of ground (cliff edges, water ripples, decorative borders). May be partially transparent.
        - [ ] **Layer 2 — Object**: Structures, doors, chests, forts, villages. Drawn above ground.
        - [ ] **Layer 3 — Units**: Unit sprites. Drawn above all terrain layers. (Managed by the Unit Sprite system, but must slot into this render order.)
        - [ ] **Layer 4 — UI / Cursor / Overlays**: Cursor, range highlights, map overlays, UI elements. Always drawn on top of everything else.
    - [ ] Each layer's tile data is stored independently. A cell may have tiles on multiple layers simultaneously.
    - [ ] A cell with no tile assigned on a given layer must render as fully transparent for that layer (revealing the layer beneath).
- [ ] **Animated Tiles**
    - [ ] Support tile animation: a tile ID may be designated as animated, in which case it cycles through a defined sequence of tile frames at a defined frame rate (in frames per game tick).
    - [ ] Animated tiles must cycle in sync globally across all instances of that tile ID on the map. Two water tiles at different map positions must always display the same animation frame simultaneously.
    - [ ] The animation cycle must be independent of the game's simulation update rate (i.e., tile animation continues at the correct visual cadence regardless of whether the game is in PLAYER_PHASE or ENEMY_PHASE).
    - [ ] Animation loop: when the last frame in the sequence is reached, the animation restarts from the first frame

seamlessly.

- [ ] **Terrain Type Data Binding**
  - [ ] Every tile ID must have a terrain type identifier bound to it. The terrain type is the data used by all gameplay systems (movement cost, defence bonus, avoid bonus, passability). It is not stored in the map layer data — it is a property of the tile ID in the tileset definition.
  - [ ] The following terrain types must be defined as the baseline set (extensible for future map design):
    - [ ] `PLAIN` — Standard traversable ground.
    - [ ] `FOREST` — Provides cover bonuses; increased movement cost for most classes.
    - [ ] `MOUNTAIN` — Impassable to most classes; accessible to fliers and mountain-specialist classes.
    - [ ] `PEAK` — Impassable to all units except flying units.
    - [ ] `WATER` / `SEA` — Impassable except for specific classes (pirates, ships) or fliers.
    - [ ] `RIVER` — Impassable except for specific classes or fliers.
    - [ ] `ROAD` — Reduced movement cost for ground units.
    - [ ] `VILLAGE` — Interactable structure tile. Grants a reward/event when a player unit moves onto it.
    - [ ] `FORT` — Defensive tile; grants defence/avoid bonus and restores HP at start of unit's turn when occupying it.
    - [ ] `GATE` — Defensive structure; typically grants the highest defensive bonus.
    - [ ] `CHEST` — Interactable object tile. Contains a loot item; becomes an `OPENED_CHEST` tile after looting.
    - [ ] `DOOR` — Impassable until opened with a key by a unit on the adjacent tile.
    - [ ] `WALL` — Impassable to all units; may be designated as `DESTRUCTIBLE_WALL` (separate state: intact vs. destroyed).
    - [ ] `THICKET` / `RUBBLE` — Difficult terrain; movement cost and bonus values vary by specific type.
    - [ ] `SAND` — Reduced avoid bonus; increased movement cost.

        - [ ] VOID / ABYSS — Out-of-bounds tile. No unit may enter. Never rendered as reachable.
    - [ ] Terrain type is the read source for: movement cost lookup (Layer 01 MT-01), terrain defence bonus (Layer 01 MT-02), terrain avoid bonus (Layer 01 MT-02), and passability rules (Layer 01 MT-03).
- [ ] **Map Data Format**
    - [ ] Maps are defined in a structured data file (not hardcoded). The data file specifies: map width and height in tiles, the tileset sheet(s) used, and per-layer tile ID arrays covering every cell in the grid.
    - [ ] The map data file must also include unit start-position data and event trigger position data for use by the event scripting system (Layer 15). These are stored in the map file but processed by their respective systems.
    - [ ] Map dimensions must support at least up to 32×32 tiles. The system must not hardcode an upper limit below 32×32.
- [ ] **Render Culling**
    - [ ] Only tiles within the current camera viewport (plus a 1-tile buffer on each edge) must be submitted for rendering each frame. Tiles outside the viewport must not be rendered.
    - [ ] The 1-tile buffer ensures no pop-in is visible when the camera scrolls.

EDGE CASES

- A map cell has a tile on Layer 2 (Object) but no tile on Layer 0 (Ground): the cell renders as transparent at Layer 0 (black void beneath). This is a map authoring error and must be flagged as a warning in the editor/loader.
- A tile ID referenced in map data does not exist in the tileset sheet: the tile must render as a visually distinct error tile (solid magenta or similar) and the missing tile reference must be logged as an error. The game must not crash.
- The game window is resized mid-session: the tile grid must re-scale to the nearest integer scale that fits the new window size and re-render correctly within one frame.
- A DESTRUCTIBLE_WALL tile is destroyed: the tile ID at that cell must be replaced with the corresponding RUBBLE tile ID at the

moment of destruction, with no gap frames where the cell is undefined.

AI-03 — AI Pathfinding System | Cat 4 | AI Systems

- [ ] **Spatial Graph Construction**
  - [ ] Construct a node graph from the map's tile grid at map load time. Each traversable tile is a node. Nodes are connected to their four orthogonal neighbours (up, down, left, right) only. Diagonal movement does not exist.
  - [ ] Each edge in the graph carries a movement cost equal to the terrain movement cost of the **destination** tile (not the origin tile). Movement cost values are read from the terrain type data bound to tile IDs (defined in ANM-03).
  - [ ] The graph is static for the duration of a chapter, with the following dynamic exceptions:
    - [ ] When a destructible wall is destroyed, its node must be added to the graph with its new terrain costs, and all four edges connecting it to its neighbours must be created. This must occur at the frame the destruction resolves.
    - [ ] When a door is opened, its node transitions from impassable to passable. Its edges must be created and activated at the frame the door opens.
    - [ ] When a unit occupies a tile, that tile's node must be flagged as OCCUPIED. The occupied status affects pathfinding (see below).
  - [ ] The pathfinding graph must be rebuilt only when a structural change occurs (wall destroyed, door opened). It must not be rebuilt every frame.
- [ ] **Movement Cost Rules**
  - [ ] Movement costs are always integers of value 1 or greater. A movement cost of 1 means the tile costs 1 movement point to enter.
  - [ ] Impassable tiles (based on terrain type and unit class) must be treated as having infinite movement cost (never enterable for that unit class). They must not appear as reachable in any movement calculation for that unit.

    - [ ] Movement cost per terrain type per class is defined in a data table. The pathfinding system reads this table, not hardcoded values. The class-specific cost lookup is described in Layer 01 (MT-01, MT-03).
- [ ] **Reachability Calculation (Move Range)**
    - [ ] Given a unit's current tile position and their Movement stat (Mov), calculate the set of all tiles that unit can reach in one action.
    - [ ] Algorithm: Dijkstra's algorithm (or equivalent optimal-cost flood fill) expanding from the unit's origin tile, accumulating movement cost along the path. A tile is reachable if the minimum total cost path to it is ≤ the unit's Mov value.
    - [ ] The reachability set must be recalculated each time it is requested (e.g., when a unit is selected). It must not be cached between turns unless the map state is confirmed unchanged.
    - [ ] The reachability calculation must respect occupied-tile rules:
        - [ ] A tile occupied by an enemy unit is **never** included in the reachable set (enemy units block passage).
        - [ ] A tile occupied by a player-allied unit may be **passed through** but is **not** a valid destination (the unit cannot stop on a tile occupied by an ally). Include it in the reachable set only as a pass-through, not as a valid destination.
        - [ ] A tile occupied by an NPC/neutral unit blocks movement in the same way as an enemy unit.
    - [ ] The output of reachability calculation is two distinct sets:
        - [ ] **Reachable Destination Set**: tiles the unit can legally stop on.
        - [ ] **Pass-Through Set**: tiles the unit can move through but not stop on (ally-occupied tiles).
    - [ ] These two sets must be kept distinct. UI systems and movement commitment logic both require them to be separate.
- [ ] **Optimal Path Calculation (Path to Destination)**
    - [ ] Given a unit's current position and a selected destination tile (from the reachable destination set), calculate the

optimal (minimum movement cost) path from origin to destination.
    - [ ] The path is returned as an ordered list of tile coordinates from origin to destination (inclusive).
    - [ ] Tie-breaking for equal-cost paths: prefer paths that minimise the number of direction changes (i.e., prefer straight lines over zigzag paths). If two paths are identical in cost and direction-change count, prefer the path that moves horizontally before vertically.
    - [ ] The optimal path must always be a valid path (every step in the path must be to an adjacent tile, movement cost along the path must not exceed the unit's Mov).
- [ ] **Attack Range Calculation**
    - [ ] Given a unit's reachable destination set and a weapon with a defined attack range (minimum range and maximum range in tiles), calculate the set of all tiles that unit can threaten (attack) from any tile in their destination set.
    - [ ] For a weapon with range [min, max], a tile T is attackable from destination tile D if the Manhattan distance between D and T is ≥ min and ≤ max.
    - [ ] The attack range set is the union of all attackable tiles across all reachable destinations.
    - [ ] The attack range calculation must correctly handle:
        - [ ] Range-1 weapons (melee): Manhattan distance = 1 only.
        - [ ] Range-2 weapons (bows, some tomes): Manhattan distance = 2 only.
        - [ ] Range 1–2 weapons: Manhattan distance = 1 or 2.
    - [ ] The attack range set is used by the UI map overlay system and by the AI attack-decision logic.
- [ ] **AI Path Execution**
    - [ ] When the AI commits a unit to move along a calculated path, the movement must be executed tile-by-tile in sequence (one tile step per movement tick), consuming the movement cost of each entered tile until the destination is reached.
    - [ ] If the path becomes blocked mid-execution (e.g., another unit moved onto a tile in the path during the same phase),

the AI unit must stop at the last valid tile before the blockage and recalculate from that position.
    - [ ] AI pathfinding may not use paths that pass through tiles occupied by enemy-faction units (enemy faction's perspective: player units block their pathing; player faction's perspective: enemy units block theirs).
- [ ] **Performance Constraint**
    - [ ] Reachability and path calculations must complete within a single frame for maps up to 32×32 tiles with up to 50 units active simultaneously. No pathfinding calculation may cause a visible frame hitch.
    - [ ] If calculations for the full enemy AI phase would cause per-frame hitching, the AI turn must process unit-by-unit with the result displayed sequentially rather than computing all units simultaneously in a single frame.

EDGE CASES

- A unit's Mov stat is 0: their reachable destination set is only their current tile. They can still attack from their current tile if they have a weapon with valid range.
- All tiles adjacent to a unit are impassable or occupied by enemies: the unit's reachable destination set is only their current tile. No crash; no infinite loop.
- A path is requested to a tile that is not in the reachable destination set (e.g., a caller passes an invalid destination): the system must return null/empty and log a warning. It must not attempt to calculate an invalid path.
- Two AI units are assigned the same destination tile in the same phase: the first unit to move claims the tile. The second unit's path must be recalculated when its turn comes. If no valid alternative destination exists, the unit waits in place.
- A door tile is opened mid-enemy-phase by a thief: all subsequent AI path calculations in that same phase must use the updated graph with the door now passable.

Layer 01 — Grid & Spatial Core

Dependency: Layer 00 must be complete before this layer begins. SYS-05 (Game State Manager), SYS-03 (Input Mapping), ANM-03 (Tile Rendering), and AI-03 (Pathfinding) must all be functional.

SYS-01 — 2D Grid Cursor System | Cat 4 | Input & Core Engine

- [ ] **Grid Coordinate Management**
  - [ ] Track cursor position as strict integer X/Y grid coordinates. X is the horizontal axis (column); Y is the vertical axis (row). Both are zero-indexed from the top-left corner of the map.
  - [ ] The cursor's grid coordinates are the authoritative spatial representation of cursor position. World-space pixel position is derived from grid coordinates and the tile size; it is never stored independently.
  - [ ] Map Boundary Clamping: Restrict all cursor coordinate updates so that X is always in the range [0, map_width - 1] and Y is always in the range [0, map_height - 1]. Any update that would move the cursor outside these bounds must be clamped to the boundary value. No exception.
- [ ] **Input Handling & Movement Logic**
  - [ ] Discrete single-press directional movement: a single press of a directional action (`CURSOR_UP`, `CURSOR_DOWN`, `CURSOR_LEFT`, `CURSOR_RIGHT`) moves the cursor exactly 1 tile in the indicated direction.
  - [ ] DAS (Delayed Auto-Shift) implementation for held directional inputs:
    - [ ] Initial delay: when a directional action is held continuously, the cursor does not begin repeating immediately. It must wait for a configurable initial delay period (default: 250 milliseconds) before the first auto-repeat movement fires.
    - [ ] Repeat rate: after the initial delay elapses, the cursor moves one tile per repeat interval. The default repeat interval is 80 milliseconds.

        - [ ] Both the initial delay and the repeat interval must be defined as named configuration constants, not magic numbers. They must be independently adjustable (e.g., for accessibility purposes).
        - [ ] If the directional action is released before the initial delay elapses, no auto-repeat movement fires. Only the single initial press movement occurs.
        - [ ] If the directional action changes (e.g., player holds RIGHT then switches to DOWN without releasing): reset the DAS timer. The initial delay starts fresh for the new direction.
    - [ ] Diagonal movement resolution: if two perpendicular directional actions are active simultaneously (e.g., `CURSOR_RIGHT` and `CURSOR_UP` both held), resolve to a single cardinal direction using the following priority: whichever directional action was pressed most recently takes priority. If both were pressed on the same frame, prefer horizontal (CURSOR_LEFT / CURSOR_RIGHT) over vertical (CURSOR_UP / CURSOR_DOWN).
    - [ ] "Fast Cursor" modifier: when the `FAST_CURSOR` logical action is held:
        - [ ] Reduce the DAS repeat interval to 25 milliseconds (default). This value is also a named configuration constant.
        - [ ] The initial DAS delay is unaffected by the fast cursor modifier.
        - [ ] Releasing `FAST_CURSOR` while a directional action is held: revert the repeat interval to the standard value. If the cursor is mid-DAS-repeat-sequence, the next repeat fires on the standard interval from that moment. Do not reset the DAS cycle.
- [ ] **Cursor Visual**
    - [ ] Render a cursor sprite centred on the tile at the cursor's current grid coordinate.
    - [ ] The cursor sprite must be drawn on Layer 4 (UI/Cursor/Overlays) as defined in ANM-03 — always above all terrain and unit layers.
    - [ ] The cursor sprite must have an idle animation (a subtle loop, e.g., pulsing or blinking outline) that plays whenever

the cursor is stationary. The animation resets when the cursor moves to a new tile.
    - [ ] When the cursor moves to a new tile, no transition animation or easing is applied. The cursor snaps instantly to the new tile's pixel position. (This is a grid-based game; cursor movement is not interpolated.)
- [ ] **Cursor State Modes**
    - [ ] The cursor operates in one of the following modes at any given time, determined by the current game/interaction state:
        - [ ] `FREE` — Default mode. The player is browsing the map with no unit selected. Moving the cursor over tiles surfaces terrain info. Pressing `CONFIRM` on a player unit tile selects that unit (transitions cursor to `UNIT_SELECTED` mode). Pressing `CONFIRM` on an empty tile or enemy tile opens the map context menu.
        - [ ] `UNIT_SELECTED` — A player unit is selected. The cursor is constrained to navigate within the unit's reachable destination set plus pass-through tiles. The reachable and non-reachable tiles are visually highlighted. Pressing `CONFIRM` on a reachable destination tile commits the unit to that destination and opens the action menu. Pressing `CANCEL` deselects the unit and returns to `FREE` mode.
        - [ ] `TARGETING` — The player is selecting an attack/staff/item target. The cursor is constrained to valid target tiles as determined by the selected action's range. Pressing `CONFIRM` on a valid target commits the action. Pressing `CANCEL` returns to the action menu without committing.
        - [ ] `LOCKED` — Cursor movement and all interaction input are fully suppressed. Used during enemy/allied phases, animations, cutscenes, and dialogue. The cursor remains visible at its last position but does not respond to input.
    - [ ] The cursor mode must be set and read through the State Manager's input context filtering (defined in SYS-03). The cursor itself does not manage state transition logic; it only enacts the behaviour defined by its current mode.

- [ ] **Cursor Memory**
  - [ ] When a unit is deselected (returning from UNIT_SELECTED to FREE mode), the cursor must return to the tile the unit occupies, not the tile the player moved the cursor to during selection.
  - [ ] When an action is cancelled from the action menu (returning from action menu to UNIT_SELECTED), the cursor must return to the unit's original tile (before any movement was committed). The unit's position is also returned to its origin tile.
  - [ ] When transitioning from BATTLE_MAP to COMBAT_ANIMATION and back, the cursor must be restored to the exact tile and mode it was in immediately before the animation began.
- [ ] **Cursor Tile Query**
  - [ ] At all times, the cursor's current grid coordinate must be queryable by any system (UI, terrain info display, combat forecast, map overlay). The query returns the integer X/Y coordinate pair.
  - [ ] At all times, the tile currently under the cursor must be queryable for: terrain type, occupying unit (if any), and all active tile flags (chest state, village state, fort, etc.).

EDGE CASES

- Cursor is at the map edge and a directional input would move it off the map: clamp to the edge. No sound effect for hitting the boundary unless a distinct "wall bump" audio cue is explicitly designed and assigned.
- CONFIRM is pressed on a tile that contains a player unit with no remaining actions (already acted this turn): the cursor must not enter UNIT_SELECTED mode. The unit's greyed-out/exhausted state is visible but they cannot be re-selected until the next player phase.
- CONFIRM is pressed on a tile that contains an enemy unit while in FREE mode: do not enter UNIT_SELECTED mode. Display the enemy unit's info panel (read-only). This is not a selection for action, only for inspection.
- The cursor is in UNIT_SELECTED mode and the selected unit is killed by a counter-attack before the player commits a

movement (edge case in specific scripted events): forcibly return the cursor to FREE mode and clear the selection. The now-dead unit's tile is no longer valid.
- Two directional inputs are pressed simultaneously on the exact same frame from keyboard (e.g., held LEFT arrow while also pressing UP): apply the horizontal-priority tie-breaking rule. Cursor moves LEFT on that frame.

SYS-02 — Camera Tracking & Panning System | Cat 4 | Input & Core Engine

- [ ] **Camera Setup**
  - [ ] Use an orthographic camera constrained to a strict 2D view with no perspective or depth. The camera's view plane is always parallel to the tile grid plane.
  - [ ] The camera's position is defined as the pixel coordinate of its top-left corner in world space. This is the only camera value that changes during gameplay — there is no zoom, no rotation, and no tilt.
  - [ ] The camera's viewport dimensions are fixed to the game's logical resolution (e.g., 240×160 logical pixels at 1× scale for GBA-style resolution, or a larger defined target resolution). The logical resolution is a project-wide constant.
- [ ] **Inner Screen Deadzone Definition**
  - [ ] Define a screen-space rectangle (the deadzone) inside the viewport. The camera remains static while the cursor moves within this rectangle.
  - [ ] Default deadzone: inset from each edge of the viewport by 3 tiles (using the logical tile size). This means if the viewport is 15 tiles wide and 10 tiles tall, the deadzone spans from tile column 3 to tile column 11 (9 tiles wide) and from tile row 3 to tile row 6 (4 tiles tall). Adjust for the actual target resolution.
  - [ ] The deadzone dimensions must be defined as named configuration constants (deadzone_margin_tiles: integer). They must not be hardcoded inline.
- [ ] **Edge-Push Scrolling**

    - [ ] When the cursor moves to a tile that would place it outside the deadzone (i.e., the cursor's screen-space position would fall outside the deadzone rectangle after the move), translate the camera in that direction by exactly 1 tile width or 1 tile height.
    - [ ] Camera translation is always in whole-tile increments. It never scrolls partial tiles.
    - [ ] Camera translation fires on the same frame as the cursor movement that triggered it. There is no delay between the cursor moving and the camera responding.
    - [ ] If the cursor moves diagonally (which resolves to a single cardinal direction per SYS-01 rules), the camera only scrolls in that resolved cardinal direction — never diagonally.
- [ ] **Map Edge Clamping**
    - [ ] The camera's top-left position must never be less than (0, 0) (cannot show content above or to the left of the map).
    - [ ] The camera's bottom-right position (top-left + viewport size) must never exceed (map_width_pixels, map_height_pixels). The camera cannot show content below or to the right of the map.
    - [ ] When the camera reaches a map edge and cannot scroll further in a direction, allow the cursor to break out of the deadzone toward that edge. The cursor must still be able to reach the edge-most tiles of the map even though the camera cannot scroll to accommodate the deadzone margin.
    - [ ] When the cursor is within the deadzone-break zone at a map edge, cursor movement toward the edge moves only the cursor (the camera stays clamped); cursor movement away from the edge behaves normally (camera scrolls if needed as cursor re-enters normal deadzone logic).
- [ ] **Pixel-Perfect Rendering Lock**
    - [ ] At every frame, before the camera position is submitted to the renderer, round the camera's X and Y position to the nearest whole integer (floor or round — must be consistent; do not alternate). This must be applied every frame without exception.
    - [ ] This ensures that tiles are always rendered at integer pixel offsets, preventing sub-pixel blurring, sprite tearing,

or jitter artefacts during camera movement.
    - [ ] The camera's internal position may be tracked at sub-pixel precision (e.g., as a float) during scroll calculations, but the value committed to the renderer must always be the integer-snapped version.
- [ ] **Forced Camera Re-centering**
    - [ ] When the game forces the cursor to a specific tile (e.g., a scripted event moves the cursor to a named character's position, or `NEXT_UNIT` cycles to a unit off-screen), the camera must re-centre on that tile immediately.
    - [ ] Re-centring: set the camera position such that the target tile is as close to the centre of the viewport as possible, subject to map-edge clamping.
    - [ ] Re-centring is instant (one frame). There is no scroll animation or easing. The camera snaps to the centred position.
- [ ] **Camera Position Queryability**
    - [ ] The camera's current top-left pixel position in world space must be queryable at any time by any system. This is used by: the cursor (to convert grid coordinates to screen-space coordinates for rendering), the UI system (to place world-anchored UI elements correctly), and the event scripting system (to determine what is currently visible to the player).

EDGE CASES

- The map is smaller than the viewport (e.g., a very small 8×6 tile test map): the camera position is fixed at (0, 0) and never scrolls. The map tiles render in the top-left of the viewport; the remainder of the viewport renders the background (outside-map colour or border). The cursor operates normally within the map bounds.
- Cursor teleports multiple tiles in one frame (e.g., `NEXT_UNIT` moves to a unit 10 tiles away): compute the final cursor position first, then perform a single re-centre camera snap. Do not process a scrolling sequence for each tile in between.
- A scripted event changes the cursor position and the camera re-centres simultaneously with a player directional input on the

same frame: the scripted event takes priority. The player input is discarded for that frame.
- The game window is resized (PC): recalculate the viewport dimensions and the deadzone rectangle from the new window size. Re-snap the camera to ensure it remains within map bounds. No frame of invalid rendering.

MT-01 — Grid-Based Movement | Cat 1 | Map & Terrain

Note: Pure FE GBA (FE7/FE8) mechanic. Cultural reskin only — names and visual framing are Indian mythology-inspired, but all movement behaviour is identical to FE GBA.

- [ ] **Movement Action Flow**
  - [ ] When a player unit is selected (cursor enters `UNIT_SELECTED` mode), immediately request and cache that unit's reachable destination set and pass-through set from AI-03.
  - [ ] Highlight the reachable destination set with a blue overlay on the affected tiles. Highlight the pass-through tiles with a visually distinct overlay (same hue, lower opacity, or a different tint) to indicate they are traversable but not stoppable.
  - [ ] Tiles outside the reachable destination set and pass-through set must not be highlighted.
  - [ ] When the player moves the cursor over a tile within the reachable destination set, display the optimal path from the unit's origin to the cursor tile as a visible path arrow overlaid on the map (a directional line connecting each tile in the path sequence).
  - [ ] The path arrow must update dynamically as the cursor moves to a different tile.
  - [ ] When the player confirms a destination tile (presses `CONFIRM` while in `UNIT_SELECTED` mode with the cursor on a valid destination):
    - [ ] Animate the unit moving tile-by-tile along the path arrow's displayed path.
    - [ ] The unit's grid position updates to each intermediate tile as the animation progresses.

        - [ ] Movement animation speed: approximately 8–10 tiles per second. This is a named configuration constant.
        - [ ] During movement animation, the cursor is locked (LOCKED mode) and the map input is suppressed.
        - [ ] When the unit reaches the destination tile, movement animation ends and the action menu opens. The cursor transitions to the appropriate mode for action selection.
    - [ ] When a unit completes a movement and then cancels their action (before finalising an attack, wait, or item use), the unit must be returned to their original tile (pre-movement), and the cursor must return to that tile. The movement is fully undone.
        - [ ] Movement undo is only available before the final action is confirmed (attack resolved, item used, or "Wait" selected). Once a final action is confirmed, movement cannot be undone.
- [ ] **Movement Cost Application**
    - [ ] Each tile entered during movement consumes movement points equal to that tile's movement cost for the moving unit's class.
    - [ ] Movement cost values are class-specific. Define a movement cost table: for every combination of (class_type × terrain_type), store the movement cost as an integer.
    - [ ] The movement cost table is read-only data — it is defined in a data file, not hardcoded. New class/terrain combinations can be added without code changes.
    - [ ] If a tile's movement cost for a given class is defined as "impassable" (infinite cost), that tile must never appear in that unit's reachable destination set or pass-through set.
    - [ ] Movement points are consumed progressively along the path. If the unit's remaining movement points exactly equal the cost to reach the destination, the destination is still valid and reachable.
- [ ] **Enemy Movement (AI-Driven)**
    - [ ] Enemy units move according to the AI pathfinding system (AI-03). Their movement follows the same

movement cost rules, terrain passability rules, and occupied-tile rules as player units.
    - [ ] Enemy unit movement is animated at the same speed as player unit movement (same configuration constant).
    - [ ] Enemy units are moved one at a time during the enemy phase. All enemy movement and actions are resolved sequentially, not simultaneously.
    - [ ] Enemy unit movement does not show the reachable destination highlight or path arrow (those are player-facing displays only). The enemy unit simply animates moving along its decided path.
- [ ] **Unit State After Acting**
    - [ ] Once a player unit completes their turn (confirms a final action: attack, item use, wait, etc.), mark that unit as "acted" for the current player phase.
    - [ ] Acted units must be displayed with a desaturated (greyed-out) sprite to visually distinguish them from units that still have available actions.
    - [ ] Acted units cannot be selected or moved again until the next player phase. Attempting to select them with `CONFIRM` must not enter `UNIT_SELECTED` mode — only the unit info panel may be opened.
    - [ ] At the start of each player phase, clear the "acted" flag on all player units. Restore all player unit sprites to their normal (fully saturated) state.
- [ ] **Allied NPC Unit Movement**
    - [ ] Allied NPC units move during the allied phase, driven by their own AI controller (Layer 10, AI-02). Movement rules, costs, and animation are identical to player and enemy unit movement.
    - [ ] The player cannot directly control allied NPC unit movement.

EDGE CASES

- A unit's movement is interrupted by a scripted event trigger on a tile mid-path (e.g., moving onto a trigger tile): the unit stops on the trigger tile, the event fires, and the movement animation does not continue past the trigger tile. The remaining path is discarded.

- A player unit attempts to move to a tile that an allied NPC unit has moved onto during the same player phase (if allied NPC movement ever occurs during player phase due to a triggered event): the tile is occupied; the player unit cannot stop there. Their reachable destination set must have been calculated before the NPC moved, so the set must be recalculated if an NPC moves during the player phase.
- Two player units are assigned movement to the same destination tile in the same phase (impossible in standard play since only one unit acts at a time in player phase, but must be guarded against in scripted events): the first unit to arrive claims the tile. The second unit's move is either cancelled or redirected to the nearest empty tile, as determined by the scripting event's logic.
- A unit's movement animation reaches a destination tile simultaneously with an enemy unit's movement animation reaching the same tile (cannot happen in normal turn order, but guards against synchronisation edge cases in allied-phase overlap): unit from the phase that owns the current active phase has priority. The other unit is displaced to the nearest valid adjacent tile.

MT-02 — Terrain Defense and Avoid Bonuses | Cat 1 | Map & Terrain

Note: Pure FE GBA (FE7/FE8) mechanic. Cultural reskin only.

Depends on: ANM-03 (terrain type data), SYS-01 (cursor tile query)

- [ ] **Terrain Bonus Data Table**
  - [ ] Define a terrain bonus table mapping each terrain type to two integer values: a Defense bonus (DEF bonus) and an Avoid bonus (AVO bonus).
  - [ ] The terrain bonus table must be defined in a data file, not hardcoded. It must be readable and modifiable without code changes.
  - [ ] Define the following default values (matching FE GBA):
    - [ ] `PLAIN` — DEF +0, AVO +0
    - [ ] `FOREST` — DEF +1, AVO +20

        - [ ] MOUNTAIN — DEF +1, AVO +30 (class-passable only)
        - [ ] PEAK — DEF +2, AVO +40 (flying only)
        - [ ] FORT — DEF +2, AVO +20
        - [ ] GATE — DEF +3, AVO +30
        - [ ] VILLAGE (occupied, before interaction) — DEF +0, AVO +0
        - [ ] ROAD — DEF +0, AVO +0
        - [ ] SAND — DEF +0, AVO +5
        - [ ] THICKET / RUBBLE — DEF +0, AVO +10
        - [ ] WATER / SEA — DEF +0, AVO +0 (passable classes only)
        - [ ] All other terrain types not listed — DEF +0, AVO +0 by default
    - [ ] The DEF bonus is added to the occupying unit's effective Defense stat when they are the defender in a combat calculation. It applies only to physical attacks (not magic attacks). See Layer 04 combat formula for application point.
    - [ ] The AVO bonus is added to the occupying unit's effective Avoid when they are the defender in a combat calculation. It applies to both physical and magical attacks. See Layer 04 combat formula for application point.
- [ ] **Terrain Bonus Application Rules**
    - [ ] The terrain bonus applies only to the unit currently **occupying** the tile (standing on it when combat is calculated). It does not apply to the attacker, even if the attacker is also on a bonus terrain tile.
    - [ ] Both the DEF bonus and the AVO bonus apply only to **incoming attacks** — when the unit is the target. They do not affect the unit's own attack damage output.
    - [ ] Flying units (fliers) receive NO terrain bonuses from any terrain type while airborne. Fliers treat all terrain as DEF +0, AVO +0. This is a class-type flag, not a per-terrain rule.
    - [ ] The terrain bonus is recalculated fresh at the time each combat calculation is resolved. It is not cached per-unit between turns.
- [ ] **Terrain Info Display**
    - [ ] When the cursor is over any tile in FREE mode or UNIT_SELECTED mode, display the terrain name, DEF

bonus, and AVO bonus for that tile in the terrain info panel (UI Layer, Layer 20).
    - [ ] The terrain name displayed must use the Indian mythology-inspired name for the terrain type (e.g., a forest tile named "Vana" rather than "Forest"). The gameplay values (DEF/AVO) are unchanged.
    - [ ] If the cursor is in LOCKED mode, the terrain info panel must not update.

EDGE CASES

- A unit is on a FORT tile: both the defence bonus and the HP restoration (defined in Layer 08's MT-07 Forts and Healing Tiles) apply simultaneously. These are independent effects.
- A flying unit lands on a GATE tile (e.g., by a scripted placement): the flier still receives no terrain bonuses. The "no terrain bonus for fliers" rule overrides the tile's bonus.
- A destructible wall is destroyed and becomes RUBBLE: the tile's terrain type updates immediately. If a unit is standing on the resulting rubble tile in the next turn's combat calculation, they receive the rubble terrain bonuses (DEF +0, AVO +10), not the wall tile's bonuses (walls are impassable and have no bonus by definition).

MT-03 — Terrain Passability by Class | Cat 1 | Map & Terrain

Note: Pure FE GBA (FE7/FE8) mechanic. Cultural reskin only.

Depends on: ANM-03 (terrain type data), AI-03 (movement cost table)

- [ ] **Class Type Definition for Passability**
    - [ ] Each unit class must have one or more movement type tags assigned. Movement type tags determine which movement cost column in the movement cost table that class uses.
    - [ ] Define the following movement type tags as the complete set:
        - [ ] INFANTRY — Standard ground unit (Soldier, Mercenary, Myrmidon, Mage, etc.)

        - [ ] ARMOURED — Heavy ground unit (Knight, General). High movement cost on most difficult terrain.
        - [ ] CAVALRY — Mounted ground unit (Cavalier, Paladin, etc.). High cost or impassable on certain terrains (mountains, peaks, forests vary).
        - [ ] FLIER — Airborne unit (Pegasus Knight, Wyvern Rider, etc.). Ignores terrain movement costs entirely — all passable terrain costs 1 movement point. Certain terrain is impassable even for fliers.
        - [ ] PIRATE / CORSAIR — Water-capable unit. Can move over water tiles at standard cost; high cost on certain land terrain.
        - [ ] THIEF — Specialised infantry with unique passability (e.g., can move through walls via lockpick; no additional terrain cost for forest; reduced mountain cost).
    - [ ] A class has exactly one movement type tag. Tags are mutually exclusive.
- [ ] **Movement Cost Table (Full)**
    - [ ] The movement cost table maps every (movement_type_tag × terrain_type) combination to an integer movement cost or the value IMPASSABLE.
    - [ ] IMPASSABLE means the tile type can never be entered by that movement class. It must be treated as having infinite cost in all pathfinding.
    - [ ] The following passability rules must be implemented as the default (matching FE GBA):
        - [ ] FLIER class: All terrain types that are not VOID/ABYSS cost 1 movement point. VOID/ABYSS is IMPASSABLE.
        - [ ] MOUNTAIN: Passable only by INFANTRY (cost 4), THIEF (cost 2), and FLIER (cost 1). CAVALRY and ARMOURED — IMPASSABLE.
        - [ ] PEAK: FLIER only (cost 1). All others — IMPASSABLE.
        - [ ] FOREST: INFANTRY (cost 2), THIEF (cost 1), CAVALRY (cost 3), ARMOURED (cost 3), FLIER (cost 1), PIRATE (cost 2).

        - [ ] WATER/SEA: PIRATE (cost 2), FLIER (cost 1). All others — IMPASSABLE.
        - [ ] RIVER: PIRATE (cost 2), FLIER (cost 1). All others — IMPASSABLE.
        - [ ] SAND: INFANTRY (cost 2), CAVALRY (cost 3), ARMOURED (cost 3), THIEF (cost 2), FLIER (cost 1), PIRATE (cost 2).
        - [ ] PLAIN, ROAD, FORT, VILLAGE, GATE, RUBBLE, THICKET: INFANTRY (cost 1 or 2 per type — match FE GBA values), CAVALRY (cost 1 or 2), ARMOURED (cost 1 or 2), THIEF (cost 1), FLIER (cost 1), PIRATE (varies — match FE GBA values).
        - [ ] WALL (intact): IMPASSABLE for all movement types.
        - [ ] DOOR (closed): IMPASSABLE for all movement types except THIEF with LOCKPICK item (handled by Layer 08 SCA-01).
        - [ ] VOID/ABYSS: IMPASSABLE for all movement types without exception.
    - [ ] The full movement cost table must be defined in a data file. The values above define the minimum required entries; the data file may include additional terrain types without code changes.
- [ ] **Runtime Passability Check**
    - [ ] All passability checks — whether a tile is passable for a given unit — must query the movement cost table using the unit's movement type tag and the tile's terrain type. There must be no hardcoded class-terrain special-cases in movement or pathfinding code.
    - [ ] When a tile returns IMPASSABLE for the querying unit's movement type, that tile is excluded from all reachability and path calculations for that unit. No unit of that movement type may ever be routed through or placed on an impassable tile.
- [ ] **Promotion Passability Update**
    - [ ] When a unit promotes to a new class (Layer 09), their movement type tag updates to the promoted class's movement type tag. All subsequent passability and movement cost calculations use the new tag. No additional

code is required for this — it is a natural consequence of the data-driven lookup.

- [ ] **Terrain Info Passability Display**
  - [ ] The terrain info panel (UI Layer, Layer 20) must display the movement cost for the selected tile based on the currently selected player unit's movement type tag (if a unit is selected). If no unit is selected, display the terrain type name and bonus values only (no class-specific cost shown).

EDGE CASES

- A flier is forced onto a `VOID`/`ABYSS` tile by a scripted event: this must be treated as an unrecoverable map authoring error. Log a critical error. Do not place the unit. The event script must be corrected.
- A unit's movement type tag is not found in the movement cost table (e.g., a new class was added but the table was not updated): treat all tiles as cost 1 for that unit and log a warning identifying the missing tag. The game must not crash.
- A `CAVALRY` unit is already standing on a `MOUNTAIN` tile (placed there by the map editor as an error): this must be treated as a map authoring error. Log a warning. Do not remove the unit at runtime. Movement calculations for that unit in their current position must treat the mountain tile as their origin and calculate reachability normally — they simply cannot step off onto another mountain tile.
- A class gains a new movement type tag via a scripted ability or transformation event (not a standard feature, but must be guarded): update the class's movement type tag in the unit's data record immediately. All subsequent calculations use the new tag.